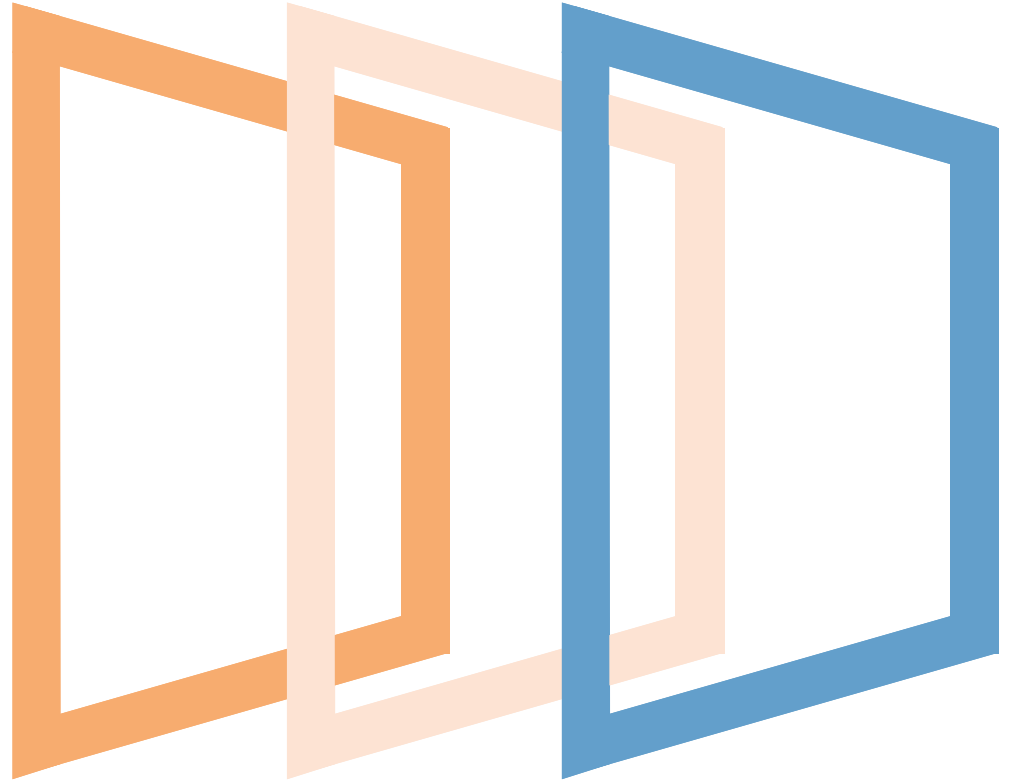


Javascript, AJAX y jQuery

Febrero 2020

minsa1t



An Indra company

Índice

01. Javascript

02. Ajax

03. jQuery

Javascript

01

¿Qué es Javascript?

- Es un lenguaje de programación que se utiliza principalmente para generar contenido web dinámico.
- Es un lenguaje interpretado que se ejecuta en el cliente, por lo que no es necesaria su compilación.
- La sintaxis de JavaScript es muy similar a la de otros lenguajes de programación como Java y C. Aunque tiene algunas particularidades
 - No se define el tipo de las variables.
 - No es necesario terminar cada sentencia con el carácter de punto y coma (;)
 - No es necesario declarar las variables, si no se declara se crea una variable global.

Ejemplo

>> Ejecutar JS01 <<

>> Generación de la letra del dni<<

El cálculo de la letra del Documento Nacional de Identidad (DNI) es un proceso matemático sencillo que se basa en obtener el resto de la división entera del número de DNI y el número 23. A partir del resto de la división, se obtiene la letra seleccionándola dentro de un array de letras.

['T', 'R', 'W', 'A', 'G', 'M', 'Y', 'F', 'P', 'D', 'X', 'B', 'N', 'J', 'Z', 'S', 'Q', 'V', 'H', 'L', 'C', 'K', 'E', 'T'];

DOM

- Los navegadores web transforman las páginas HTML en un árbol de nodos, que puede ser parseada con herramientas DOM.

```
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Página sencilla</title>
</head>

<body>
<p>Esta página es <strong>muy sencilla</strong></p>
</body>
</html>
```

Párrafo, genera dos nodos: el primer nodo es de tipo Element y representa la etiqueta <p> y el segundo nodo es de tipo Text y representa el contenido textual de la etiqueta <p>.

Acceso directo a los nodos

- `getElementsByTagName()`

```
var parrafos = document.getElementsByTagName("p");
```

- `getElementsByTagName ()` → Radio button!!

```
var parrafoEspecial = document.getElementsByTagName("especial");
```

```
<p name="especial">...</p>
```

- `getElementById()`

```
var parrafoEspecial = document.getElementById("especial");
```

```
<p id="especial">...</p>
```

Creación de nodos

- Creación de un nodo de tipo Element que represente al elemento.

```
var parrafo = document.createElement("p");
```

- Creación de un nodo de tipo Text que represente el contenido del elemento

```
var contenido = document.createTextNode("Hola Mundo!");
```

- Añadir el nodo Text como nodo hijo del nodo Element.

```
parrafo.appendChild(contenido);
```

- Añadir el nodo Element a la página, en forma de nodo hijo del nodo correspondiente al lugar en el que se quiere insertar el elemento.

```
document.body.appendChild(parrafo);
```


Eliminar nodos

- Para eliminar el nodo sólo se tiene que hacer una llamada a `removeChild` pasándole el nodo que queremos eliminar.

```
var parrafo = document.getElementById("provisional");  
parrafo.parentNode.removeChild(parrafo)
```

Acceso directo a los atributos

- Una vez se ha obtenido el nodo, es fácil acceder a sus atributos e incluso modificarlos como nosotros queramos:

```
<a id="enlace" href="http://www...com">Enlace</a>
```

```
var enlace = document.getElementById("enlace");  
alert(enlace.href); // muestra http://www...com
```

>> Ejecutar JS02 <<

Eventos

- En JS lo habitual es que se ejecuten las funciones cuando se produzca un evento: seleccionar un botón, una opción del desplegable, cambiar el foco de un campo...
- Para ello se usa los atributos de HTML, y la función que se ejecuta recibe el propio evento (en un parámetro que no se especifica, pero que está ahí, sólo se tiene que declarar).

>> Información detallada de eventos DOM <<

>> Ejecutar JS03 <<

Formularios

- Cuando se carga una página web, el navegador crea automáticamente un array llamado `forms` y que contiene la referencia a todos los formularios de la página.

`document.forms[0];`

- También genera un array de los elementos que compone cada formulario:

`document.forms[0].elements[0];`

- La principal utilidad de JavaScript en el manejo de los formularios es la validación de los datos introducidos por los usuarios

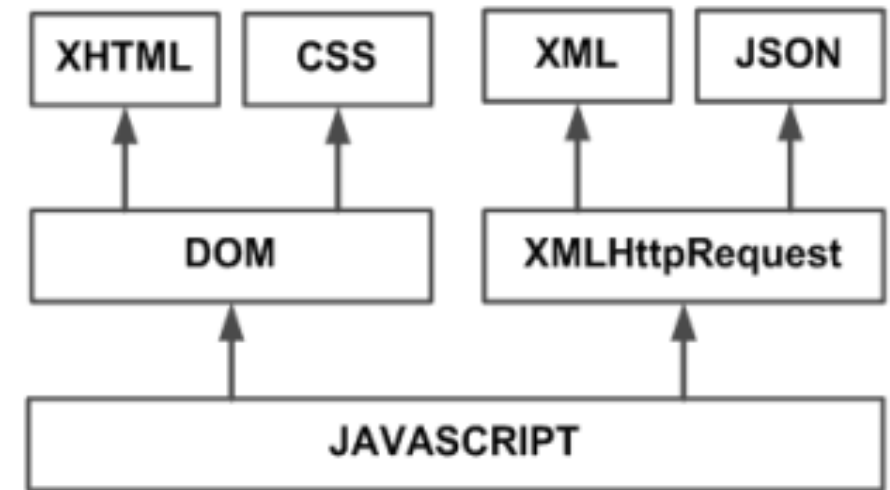
>> Adaptar el ejemplo de generación de DNI <<

Ajax

02

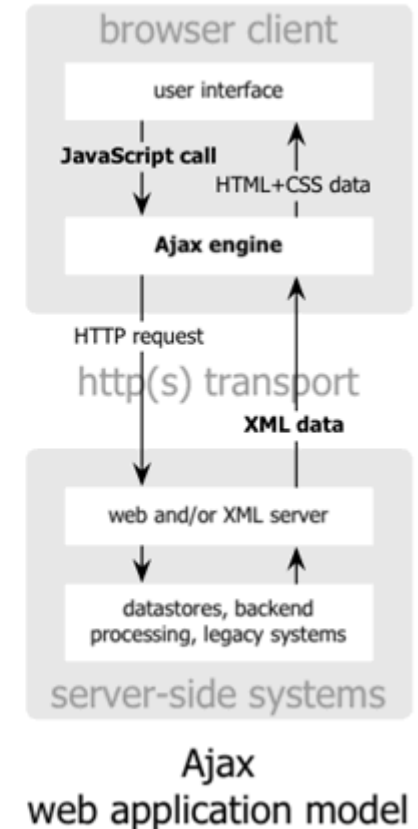
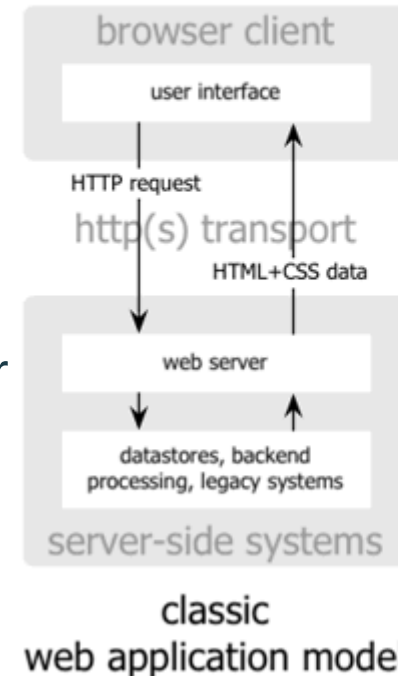
¿Qué es Ajax?

- El término AJAX se presentó por primera vez en un artículo en 2005 y se definió como:
Ajax no es una tecnología en sí mismo. En realidad, se trata de varias tecnologías independientes que se unen de formas nuevas y sorprendentes:
- XHTML y CSS, para crear una presentación basada en estándares.
- DOM, para la interacción y manipulación dinámica de la presentación.
- XML, XSLT y JSON, para el intercambio y la manipulación de información.
- XMLHttpRequest, para el intercambio asíncrono de información.
- JavaScript, para unir todas las demás tecnologías.



¿Y cómo funciona Ajax?

- En las aplicaciones web tradicionales, las acciones del usuario en la página desencadenan llamadas al servidor. Una vez procesada la petición del usuario, el servidor devuelve una nueva página HTML.
- En las aplicaciones AJAX, las peticiones más simples no requieren intervención del servidor, por lo que la respuesta es inmediata. Si la interacción requiere una respuesta del servidor, la petición se realiza de forma asíncrona mediante AJAX.



Javascript avanzado – Arrays asociativos y notación por puntos

- Un array asociativo es aquel en el que cada elemento no está asociado a su posición numérica dentro del array, sino que está asociado a otro valor específico.

```
var elArray = new Array();
```

```
elArray['primero'] = 1;
```

```
elArray['segundo'] = 2;
```



```
alert(elArray['primero']);
```

 ← Devuelve 1

```
alert(elArray[0]);
```

 ← Devuelve undefined

```
var elArray = new Array();
```

```
elArray.primero = 1;
```

```
elArray.segundo = 2;
```

```
alert(elArray['primero']);
```

```
alert(elArray.primero);
```


Javascript avanzado – Arrays asociativos y notación por puntos

- Además de las propiedades, los métodos de los objetos también se pueden definir mediante la notación de puntos, asignándoles funciones anónimas o “normales”

```
elObjeto.muestraId = function() {  
    alert("El ID del objeto es " + this.id);  
}
```

Función
anónima



```
function obtieneId() {  
    return this.id;  
}
```

Función
normal

```
elObjeto.obtieneId = obtieneId;
```

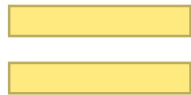
Javascript avanzado – Función call() y apply()

- Ambos métodos permiten ejecutar una función como si fuera un método de otro objeto. La única diferencia entre los dos métodos es la forma en la que se pasan los argumentos a la función.

```
function miFuncion(x) {  
    return this.numero + x;  
}
```

```
var elObjeto = new Object();  
elObjeto.numero = 5;
```

```
var resultado = miFuncion.call(elObjeto, 4);  
alert(resultado);
```



```
function miFuncion(x) {  
    return this.numero + x;  
}
```

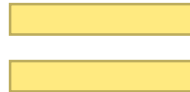
```
var elObjeto = new Object();  
elObjeto.numero = 5;
```

```
var resultado = miFuncion.apply(elObjeto, [4]);  
alert(resultado);
```

Javascript avanzado – JSON

- El formato JSON permite representar estructuras de datos (arrays) y objetos (arrays asociativos) en forma de texto.

```
var modulos = new Array();  
modulos.titulos = new Array();  
modulos.titulos.rss = "Lector RSS";  
modulos.titulos.email = "Gestor de email";  
modulos.titulos.agenda = "Agenda";
```



```
var modulos = new Array();  
modulos.titulos = {rss: "Lector RSS", email: "Gestor de email", agenda: "Agenda"};
```

>> Ejecutar JS04 <<

Empezando con Ajax

- Todas las aplicaciones realizadas con técnicas de AJAX deben instanciar en primer lugar el objeto XMLHttpRequest, el cual nos permitirá realizar peticiones en segundo plano.
- La propiedad onreadystatechange del objeto XMLHttpRequest permite indicar qué función se va a ejecutar cuando la aplicación reciba la respuesta del servidor.
- Se crea la petición y se envía.
- Finalmente, al ejecutarse la función se comprueba que se ha recibido la respuesta del servidor y que es correcta (status=200)

>> Ejecutar JS05 <<

Empezando con Ajax

- Todas las aplicaciones realizadas con técnicas de AJAX deben instanciar en primer lugar el objeto XMLHttpRequest, el cual nos permitirá realizar peticiones en segundo plano.
- La propiedad onreadystatechange del objeto XMLHttpRequest permite indicar qué función se va a ejecutar cuando la aplicación reciba la respuesta del servidor.
- Se crea la petición y se envía.
- Finalmente, al ejecutarse la función se comprueba que se ha recibido la respuesta del servidor y que es correcta (status=200)

>> Ejecutar JS05 <<

Ejercicio

>> Obtener datos de las peticiones <<

A partir de la página web proporcionada, añadir el código JavaScript necesario para que:

1. Al cargar la página, el cuadro de texto debe mostrar por defecto la URL de la propia página.
2. Al pulsar el botón "Mostrar Contenidos", se debe descargar mediante peticiones AJAX el contenido correspondiente a la URL introducida por el usuario. El contenido de la respuesta recibida del servidor se debe mostrar en la zona de "Contenidos del archivo".
3. En la zona "Estados de la petición" se debe mostrar en todo momento el estado en el que se encuentra la petición (No inicializada, cargando, completada, etc.)
4. Mostrar el contenido de todas las cabeceras de la respuesta del servidor en la zona "Cabeceras HTTP de la respuesta del servidor".
5. Mostrar el código y texto de estado de la respuesta del servidor en la zona "Código de estado".

>> Ejecutar JS07, JS08 <<

jQuery

03

¿Qué es jQuery?

- jQuery es una librería JS y como tal, debe ser descargado el JS para poder acceder a sus métodos.

>> Descargar librería jQuery <<

- En cada página que lo requiera, habrá que agregar:
`<script type="text/javascript" src="jquery.js"></script>`

Selección de elementos

- La función básica de jQuery y una de las más útiles es la "función dolar": `$()` y equivale a la función `document.getElementById()`.
 - Selección de elementos en base a su id → `$('#myId')`
 - Selección de elementos en base al nombre de clase → `$('.div.myClass');`
 - Selección de elementos por su atributo → `$( 'input[name=first_name]');`
 - Selección de elementos en forma de selector CSS → `$('#contents ul.people li');`
 - Pseudo-selectores
 - selecciona el primer elemento `<a>` con la clase 'external' → `$('.a.external:first');`
 - selecciona todos los elementos `<tr>` impares de una tabla → `$('.tr:odd');`
- Para comprobar si se ha realizado una selección correcta, se debe utilizar la propiedad `length`:
`if ($('#div.foo').length) { ... }`
- Para guardar las selecciones en una variable, se ejecuta la siguiente instrucción:
`var $divs = $('div');`

Ejercicio

>> Resolver JS09 <<

- Seleccionar todos los elementos div que poseen la clase module.
- Averiguar cuantos elementos en la página están ocultos (ayuda: .length)
- Averiguar cuantas imágenes en la página poseen el atributo alt.
- Seleccionar todas las filas impares del cuerpo de la tabla.

Trabajar con las selecciones

- Los métodos get y set, se llaman igual, la diferencia es que en el set le pasamos el nuevo valor que queremos establecer.
- Existen los siguientes métodos para manipular:
 - \$.fn.html Obtiene o establece el contenido HTML de un elemento.
 - \$.fn.text Obtiene o establece el contenido en texto del elemento; en caso se pasarle como argumento código HTML, este es despojado.
 - \$.fn.attr Obtiene o establece el valor de un determinado atributo.
 - \$.fn.width Obtiene o establece el ancho en pixeles del primer elemento de la selección como un entero.
 - \$.fn.height Obtiene o establece el alto en pixeles del primer elemento de la selección como un entero.
 - \$.fn.position Obtiene un objeto con información sobre la posición del primer elemento de la selección, relativo al primer elemento padre posicionado. Este método es solo obtenedor.
 - \$.fn.val Obtiene o establece el valor (value) en elementos de formularios.

```
$('#myDiv p:first')
```

```
.html('Nuevo <strong>primer</strong> párrafo');
```

Trabajar con las selecciones

- Se puede crear nuevos elementos con la función \$():

```
$('#<p>Un nuevo párrafo</p>');  
$('#<li class="new">nuevo item de la lista</li>');
```

- Un nuevo elemento con atributos:

```
$('#<a/>', {  
    html : 'Un <strong>nuevo</strong> enlace',  
    'class' : 'new',  
    href : 'foo.html'  
});
```

- Cuando se crea un elemento, no se añade inmediatamente a la página, sino que se debe hacer en conjunto con un método.

```
var $myNewElement = $('#<p>Nuevo elemento</p>');  
$myNewElement.appendTo('#content');
```

Trabajar con las selecciones

- Se pueden manipular los atributos a través de \$.fn.attr

```
$('#myDiv a:first').attr('href', 'newDestination.html');
```

```
$('#myDiv a:first').attr({  
    href : 'newDestination.html',  
    rel  : 'super-special'  
});
```

Ejercicio

>> Resolver JS10 <<

- Añadir 5 nuevos ítems al final de la lista desordenada #myList.

Ayuda: `for (var i = 0; i<5; i++) { ... }`

- Remover los ítems impares de la lista.

Ayuda: función `remove()`

- Añadir otro elemento h2 y otro párrafo al último div.module.

- Añadir otra opción al elemento select; darle a la opción añadida el valor Wednesday.

Eventos

- Una de las funciones mejor consideradas es la que se encarga de indicar que la página ha sido cargada:

```
$(document).ready(function() {  
    ...  
});
```

- La gran ventaja es que la aplicación no espera a que se carguen todos los elementos de la página, sino que sólo espera a que se haya descargado el contenido HTML de la página, con lo que el árbol DOM ya está disponible para ser manipulado.
- El uso más habitual de las funciones de evento es el de establecer su función manejadora:

```
// Establece la función manejadora del evento 'onclick'  
// a todos los párrafos de la página  
$('p').click(function() {  
    alert($(this).text());  
});
```

Funciones para Ajax

- El método principal para realizar peticiones AJAX es \$.ajax()

```
$.ajax({  
    url: '/ruta/hasta/pagina.php',  
    type: 'POST',  
    async: true,  
    data: 'parametro1=valor1&parametro2=valor2',  
    success: procesaRespuesta,  
    error: muestraError  
});
```


Funciones para Ajax

- También se han definido las funciones \$.get() y \$.post() para realizar peticiones GET y POST. Su sintaxis es: \$.get(url, datos, funcionManejadora);

```
// Petición GET simple
```

```
$.get('/ruta/hasta/pagina.php');
```

```
// Petición GET con envío de parámetros y función que
```

```
// procesa la respuesta
```

```
$.get('/ruta/hasta/pagina.php',
```

```
  { articulo: '34' },
```

```
  function(datos) {
```

```
    alert('Respuesta = '+datos);
```

```
  });
```

Funciones para Ajax

- La función \$.load() inserta el contenido de la respuesta del servidor en el elemento de la página que se indica.

```
<div id="info"></div>
```

```
// Con jQuery
```

```
$('#info').load('/ruta/hasta/pagina.php');
```

¡Gracias!

Presentación:

Vanessa Andres Blas

vablas@minsait.com

Parc tecnològic de Gardeny, H3, 1^a

25003 Lleida

Lleida España

T +34 973 19 35 16

E 240316

www.minsait.com

minsait

An Indra company

minsait

Mark Making the way forward

An Indra company